



Вычислительная геометрия и алгоритмы компьютерной графики

Лекция №9: Текстуры

к.ф.-м.н., доцент

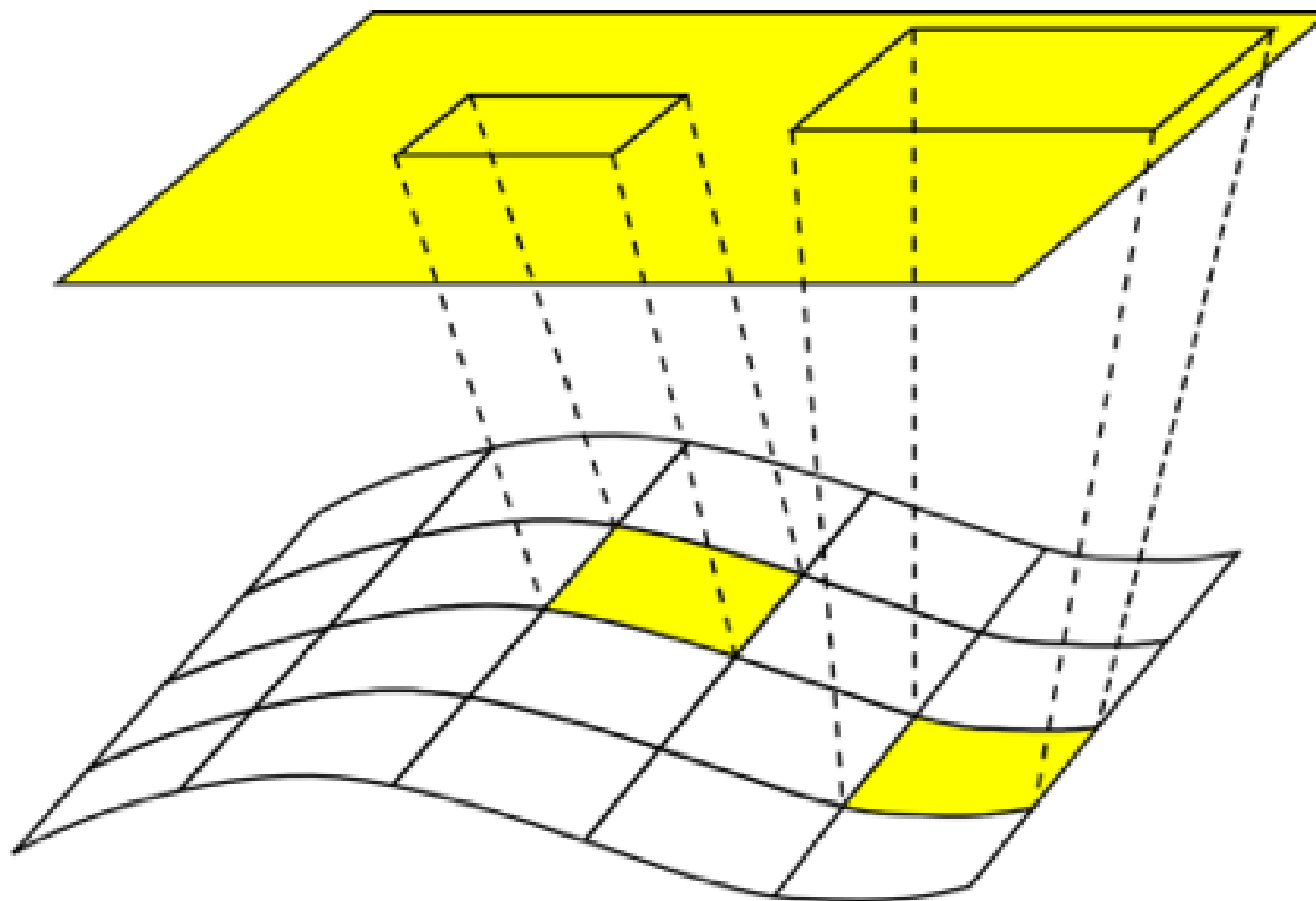
Рябинин Константин Валентинович,
kostya.ryabinin@gmail.com

ст. преподаватель

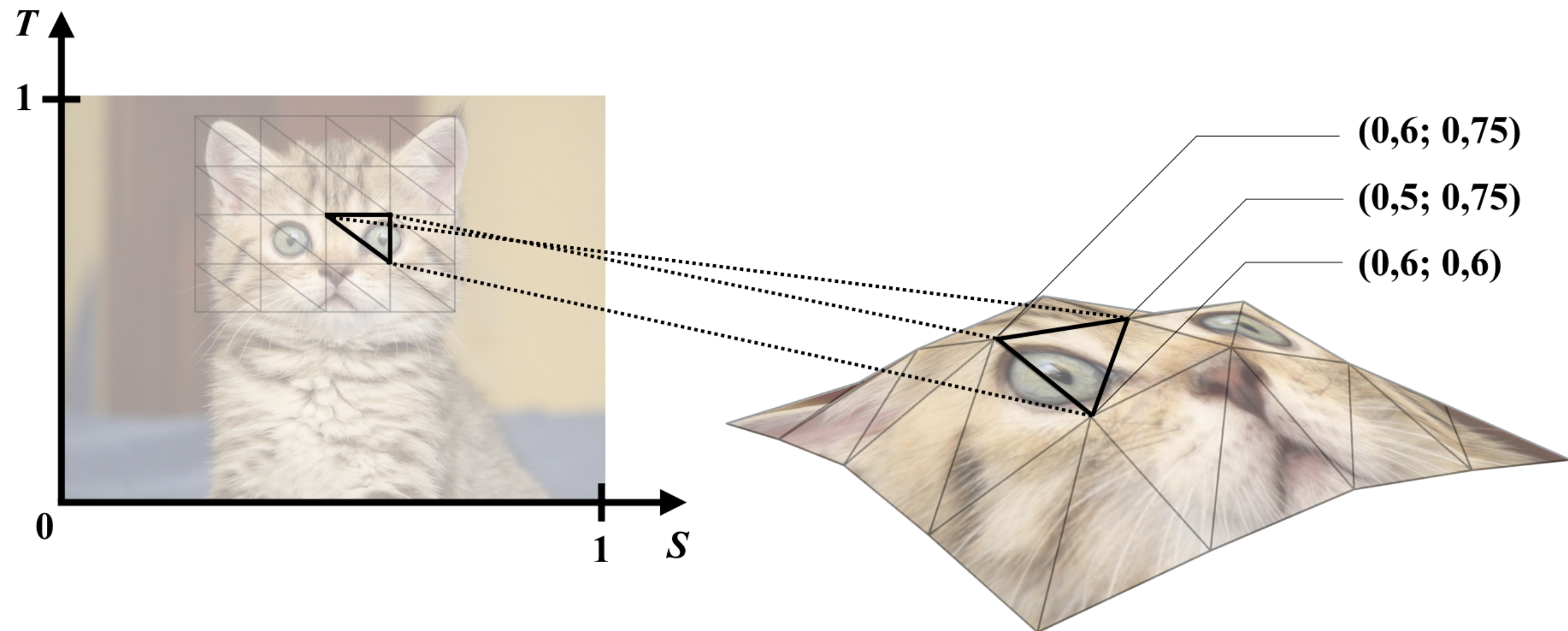
Лабутин Иван Александрович,
linuxrf@gmail.com

Пермь – 2025

Двумерная текстура – растровое изображение, накладываемое на поверхность полигонов, из которых состоят 3D-модели, для придания ей цвета, окраски и иллюзии рельефа

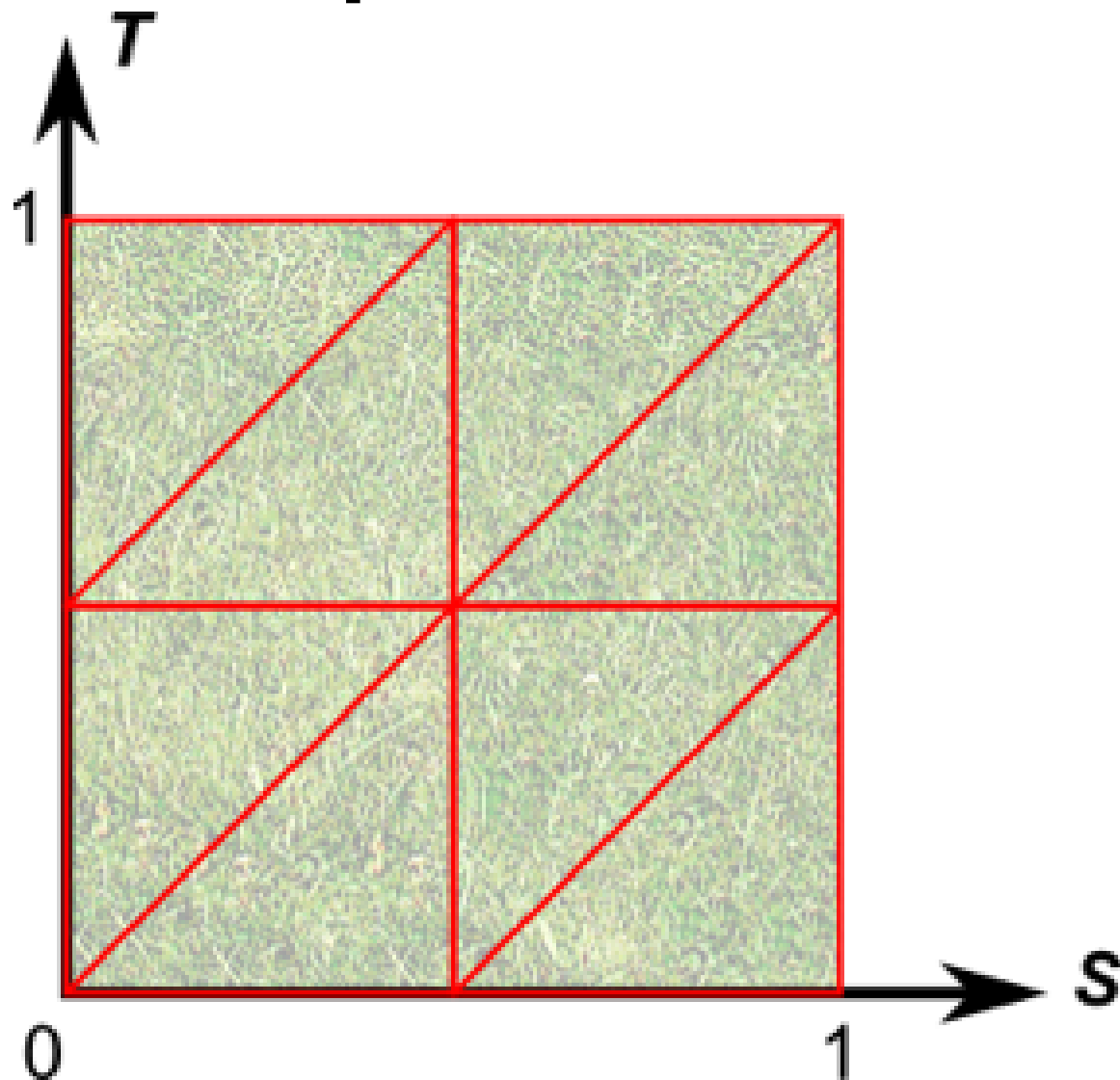


Текстурные координаты вершины – координаты проекции данной вершины на изображение

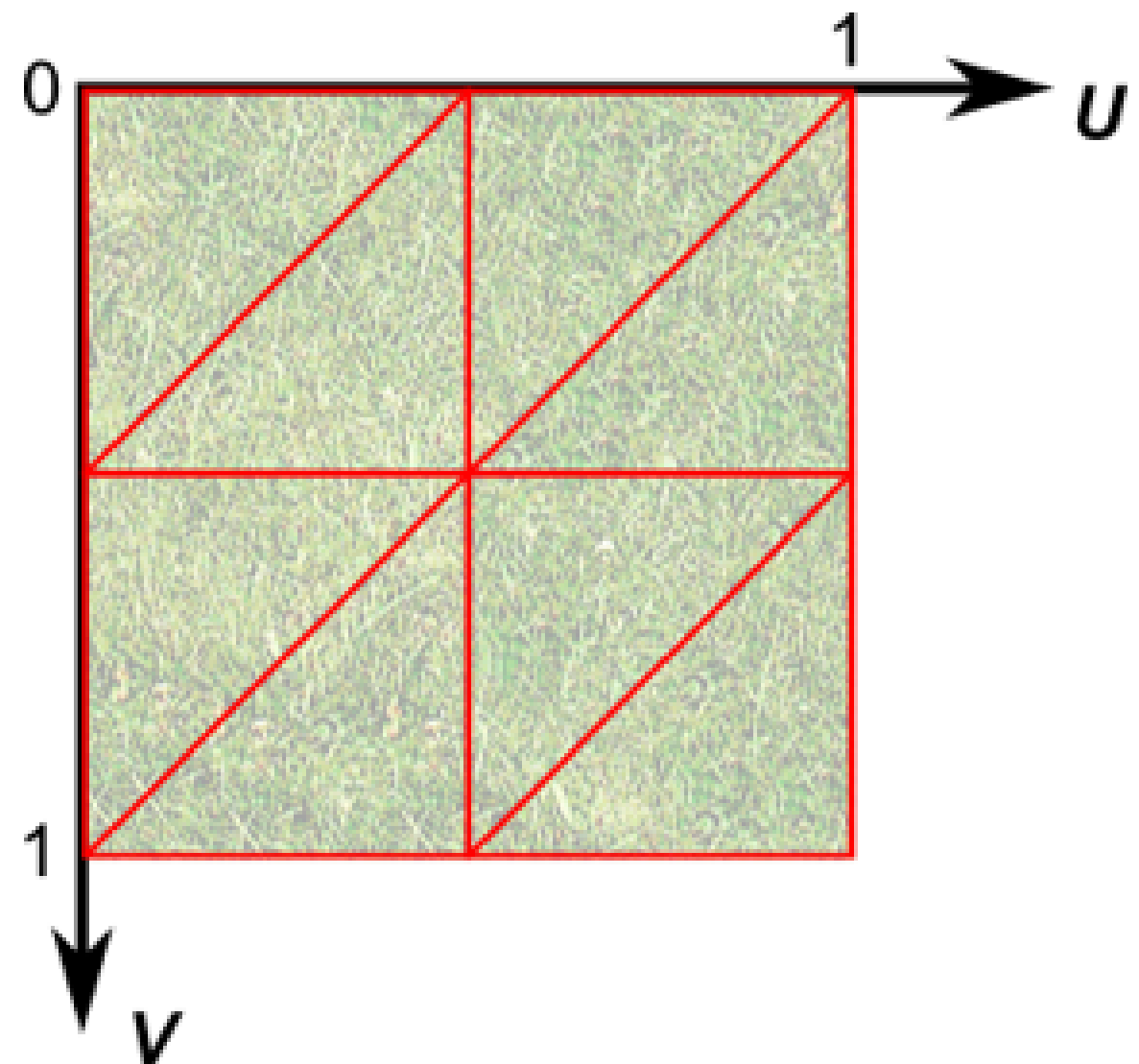


**Система координат определяется соглашением
внутри используемой спецификации**

OpenGL



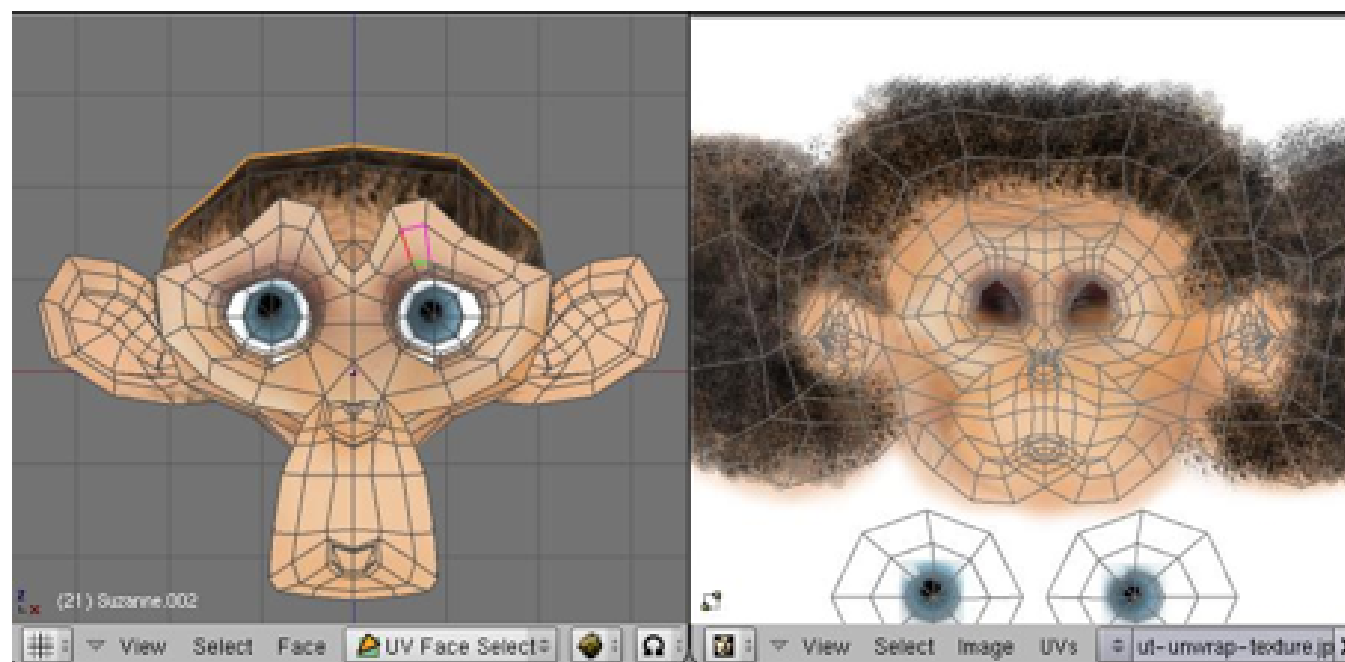
Direct3D



Карта текстуры для 3D-модели – это множество координат для каждой вершины данной модели

В частных случаях карта может быть сгенерирована для модели автоматически, но в общем случае это НЕВОЗМОЖНО

Поэтому, как правило, карта текстуры создаётся высокоуровневыми средствами (в 3D-редакторах) и загружается в программу вместе с моделью



1. Аналитическое получение (процедурная текстура)

1. Простые шаблоны:

1. Повторяющиеся регулярные узоры

2. Штриховки

3. . . .

2. Фракталы

3. Графики и изолинии функций

2. Загрузка из файла (наперёд заданный растр)

3. Рендеринг

1. Без постобработки

2. С постобработкой

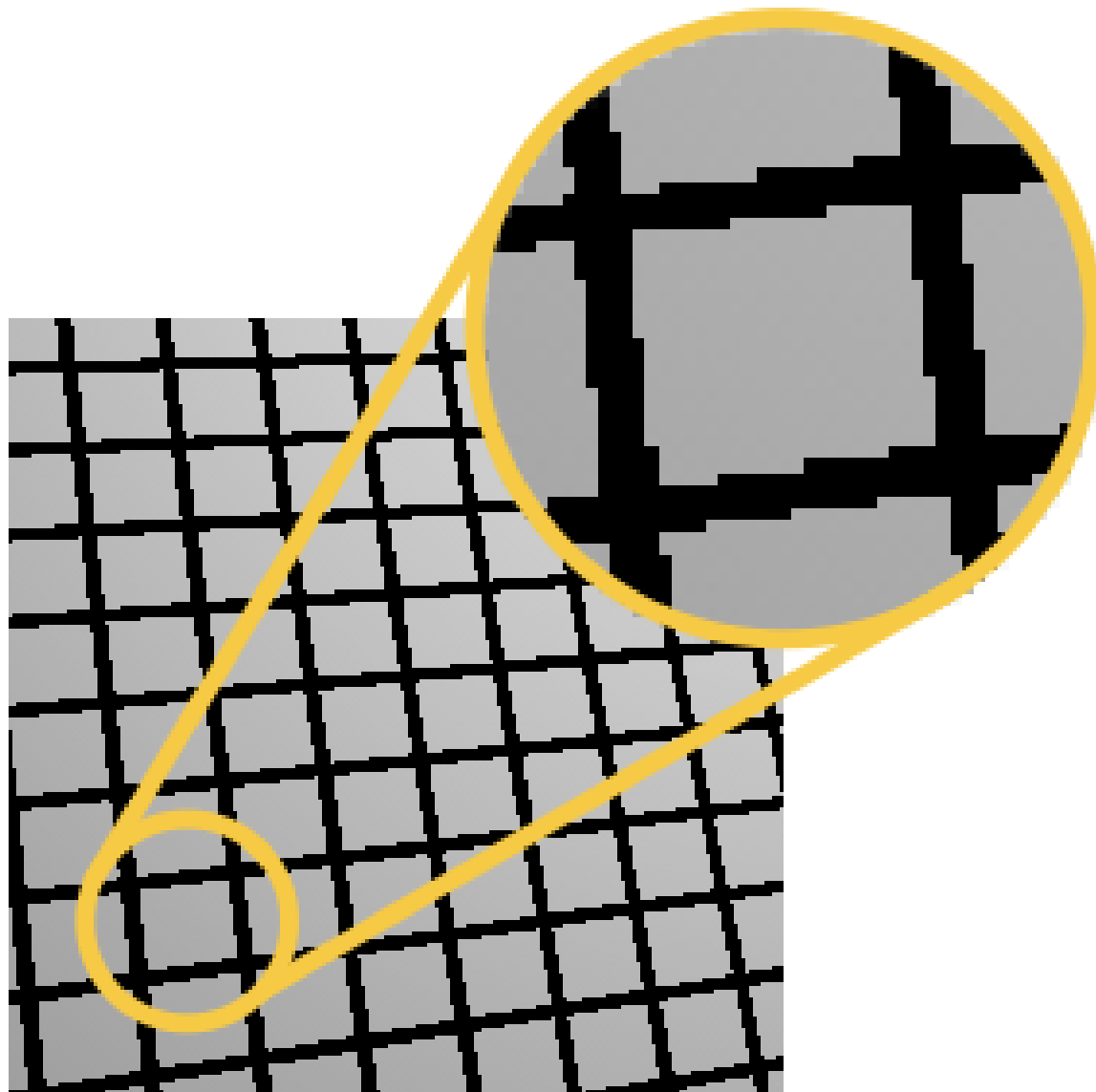
- 1. Окраска**
- 2. Управление свойствами материала
(diffuse / specular mapping)**
- 3. Обеспечение иллюзии рельефа (normal mapping)**
- 4. Управление формой (displacement mapping)**
- 5. Управление отображением (masking)**
- 6. Управление физическими характеристиками
(float-текстуры)**
- 7. ...**

Задача: зная интерполированные текстурные координаты в данной точке полигона извлечь цвет из текстуры

Проблема: текстурные координаты (u, v) являются вещественными и меняются в отрезке $[0; 1]$, а изображение имеет размер $m \times n$, где m, n – натуральные числа.

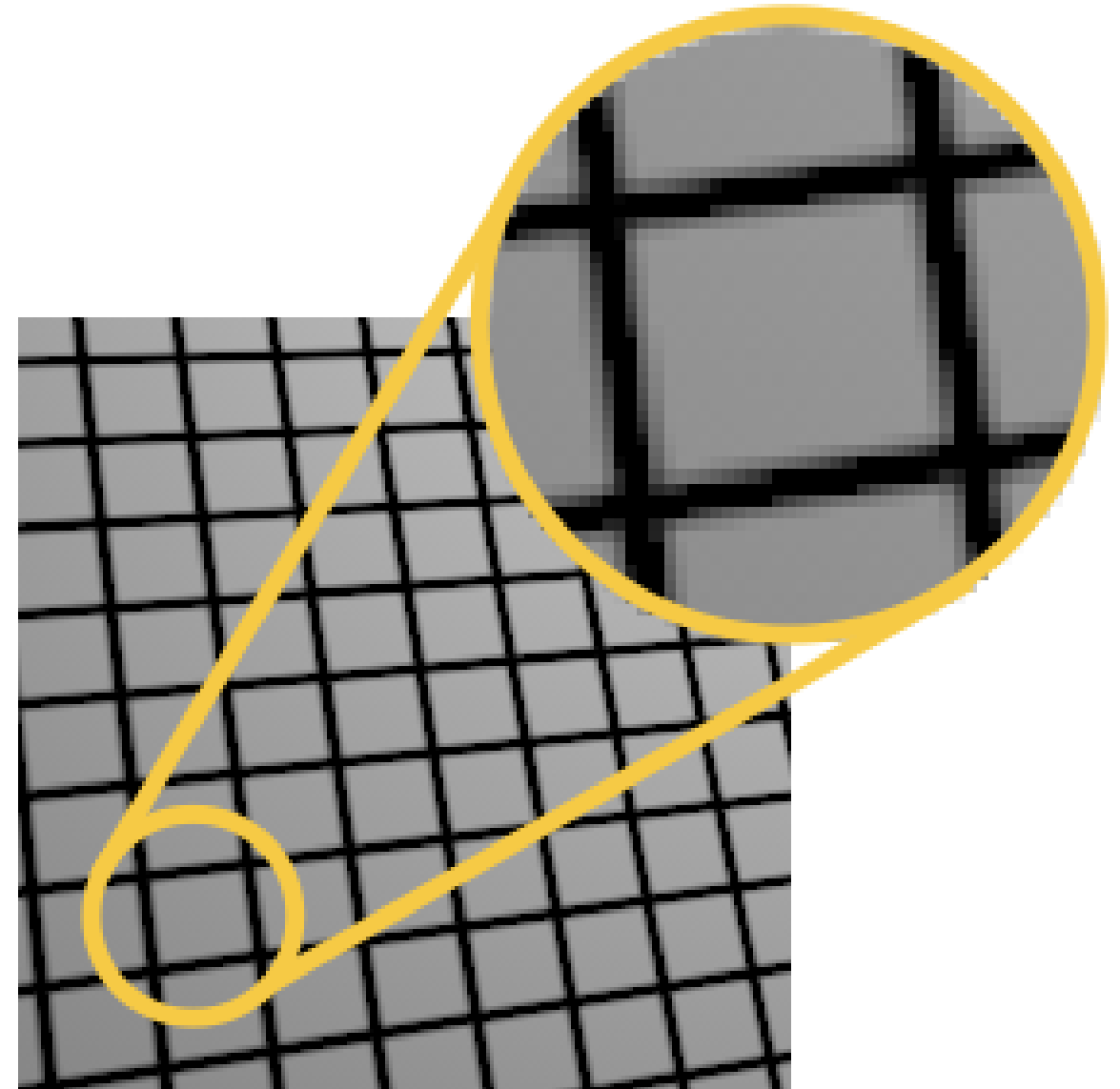
⇒ Постоянно возникает ситуация, когда (mu, nv) – далеко не целые числа, а значит не адресуют конкретный пиксель текстуры

⇒ Надо каким-то образом осуществлять выборку цвета из текстуры по дробным координатам



GL_NEAREST

**Алгоритм взятия
ближайшего соседа**



GL_LINEAR

**Билинейная
фильтрация**

```
Color getBilinearFilteredPixelColor(Texture tex, double u, double v)
{
    u = u * tex.size - 0.5;
    v = v * tex.size - 0.5;

    int x = floor(u);
    int y = floor(v);

    double u_ratio = u - x;
    double v_ratio = v - y;
    double u_opposite = 1 - u_ratio;
    double v_opposite = 1 - v_ratio;

    return (tex[x][y] * u_opposite + tex[x+1][y] * u_ratio) * v_opposite +
           (tex[x][y+1] * u_opposite + tex[x+1][y+1] * u_ratio) * v_ratio;
}
```

**Бывают и другие, более сложные способы
фильтрации, дающие более качественный результат**

**Но на практике часто используется именно
билинейная фильтрация**

Она имеет аппаратную поддержку

В идеале текстура $m \times n$ накладывается на поверхность, проекция которой на экран имеет размер $m \times n$

Но на практике это далеко не всегда так

В результате требуются
минификация (уменьшение)
или
магнификация (увеличение)
текстуры при наложении

Размер текстуры: 32 x 32

Наложение:

32×32



GL_LINEAR

512×512



GL_NEAREST

512×512



GL_LINEAR

Размер текстуры: 512 x 512

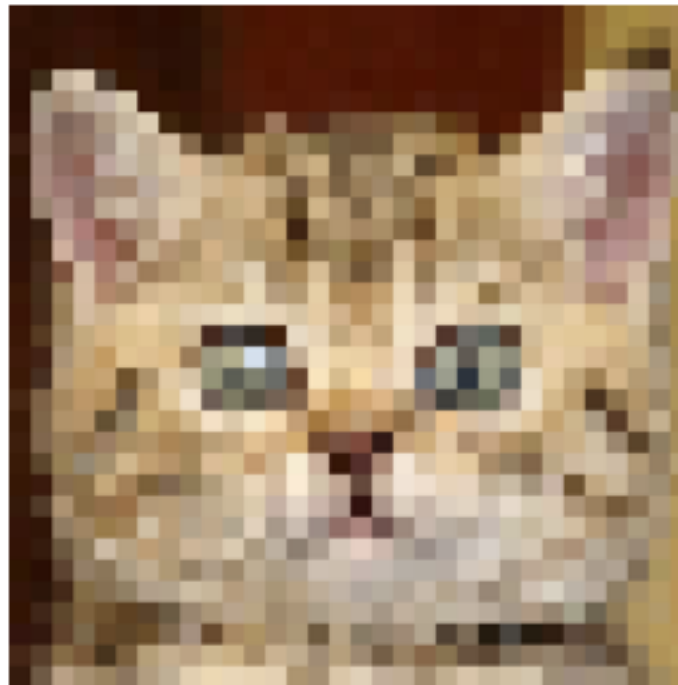
Наложение:

512 × 512



GL_NEAREST

32 × 32



GL_NEAREST

32 × 32



GL_LINEAR

**Чтобы по возможности уравнивать размер текстуры
с размером проекции объекта,
используют мипмэппинг (MIP – Multum In Parvo)**

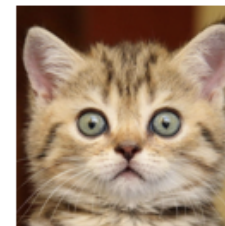
Чтобы по возможности уравнивать размер текстуры с размером проекции объекта, используют мипмэппинг (MIP – Multum In Parvo)



512 × 512



256 × 256



128 × 128



64 × 64



32 × 32



16 × 16



8 × 8



4 × 4



2 × 2



1 × 1

MIP-пирамида

Чтобы по возможности уравнивать размер текстуры с размером проекции объекта, используют мипмэппинг (MIP – Multum In Parvo)

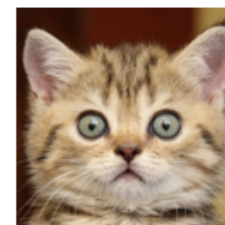
Уменьшение размеров делается **однократно** более качественным (и медленным) алгоритмом, чем билинейная фильтрация



512×512



256×256



128×128



64×64



32×32



16×16



8×8



4×4



2×2



1×1

МІР-пирамида

Чтобы по возможности уравнивать размер текстуры с размером проекции объекта, используют мипмэппинг (MIP – Multum In Parvo)

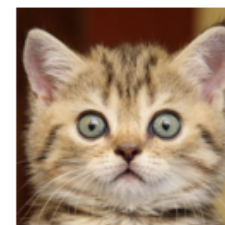
Уменьшение размеров делается **однократно** более качественным (и медленным) алгоритмом, чем билинейная фильтрация



512×512



256×256



128×128



64×64



32×32



16×16



8×8



4×4



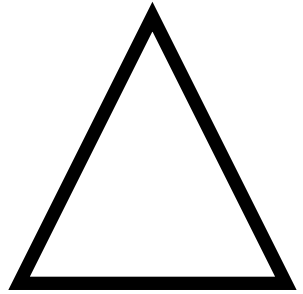
2×2

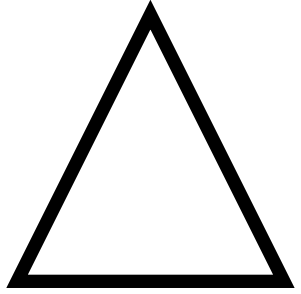


1×1

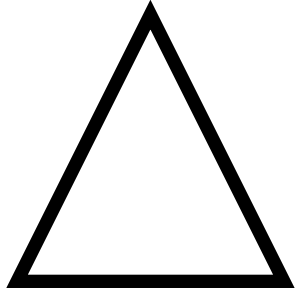
Расход памяти увеличивается всего на 1/3

MIP-пирамида



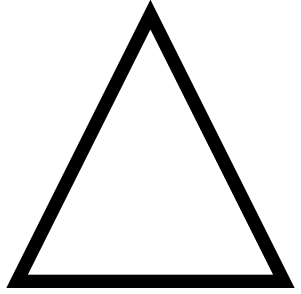


$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$



$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

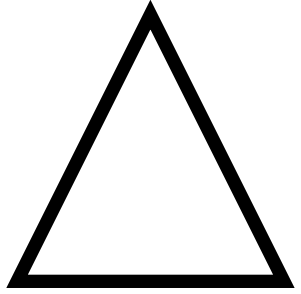
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$



$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

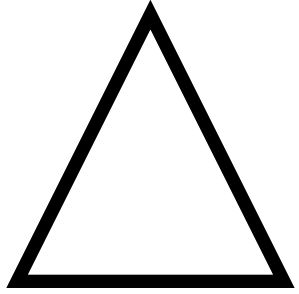


$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$



$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

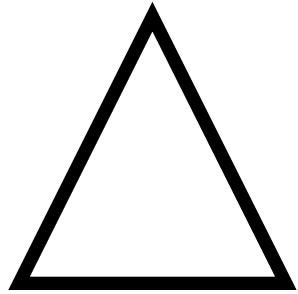
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$



$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

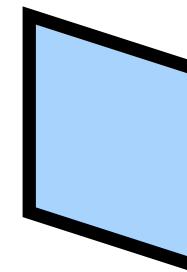
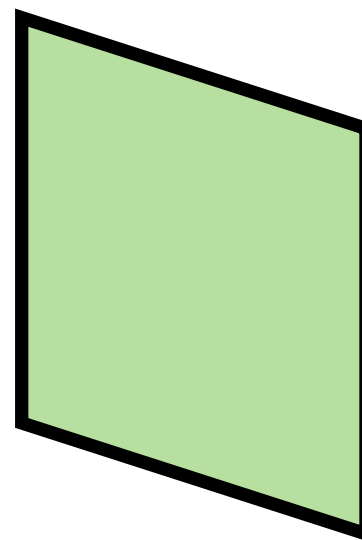
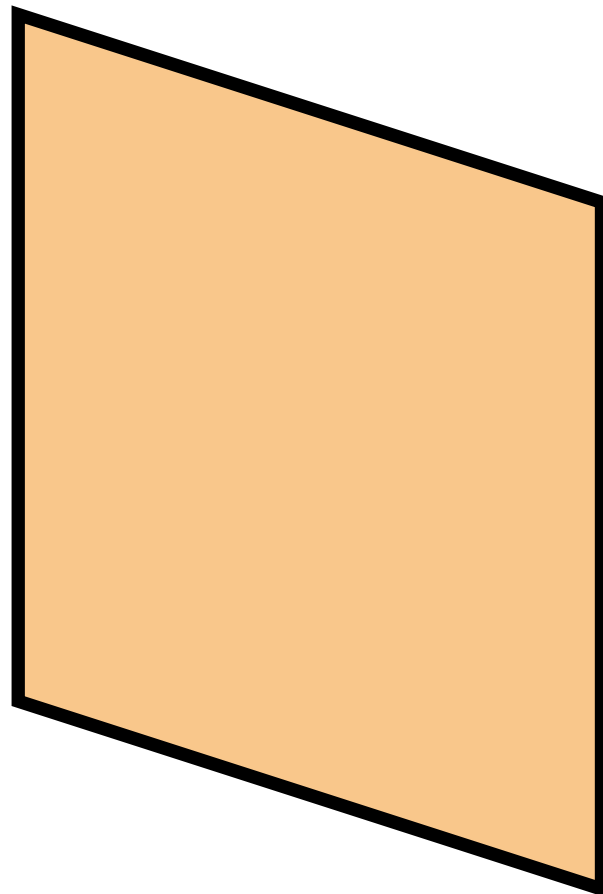
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

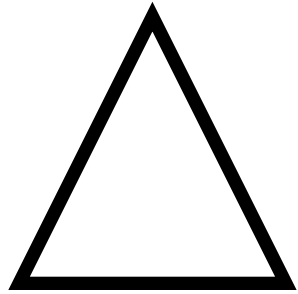
$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$





$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

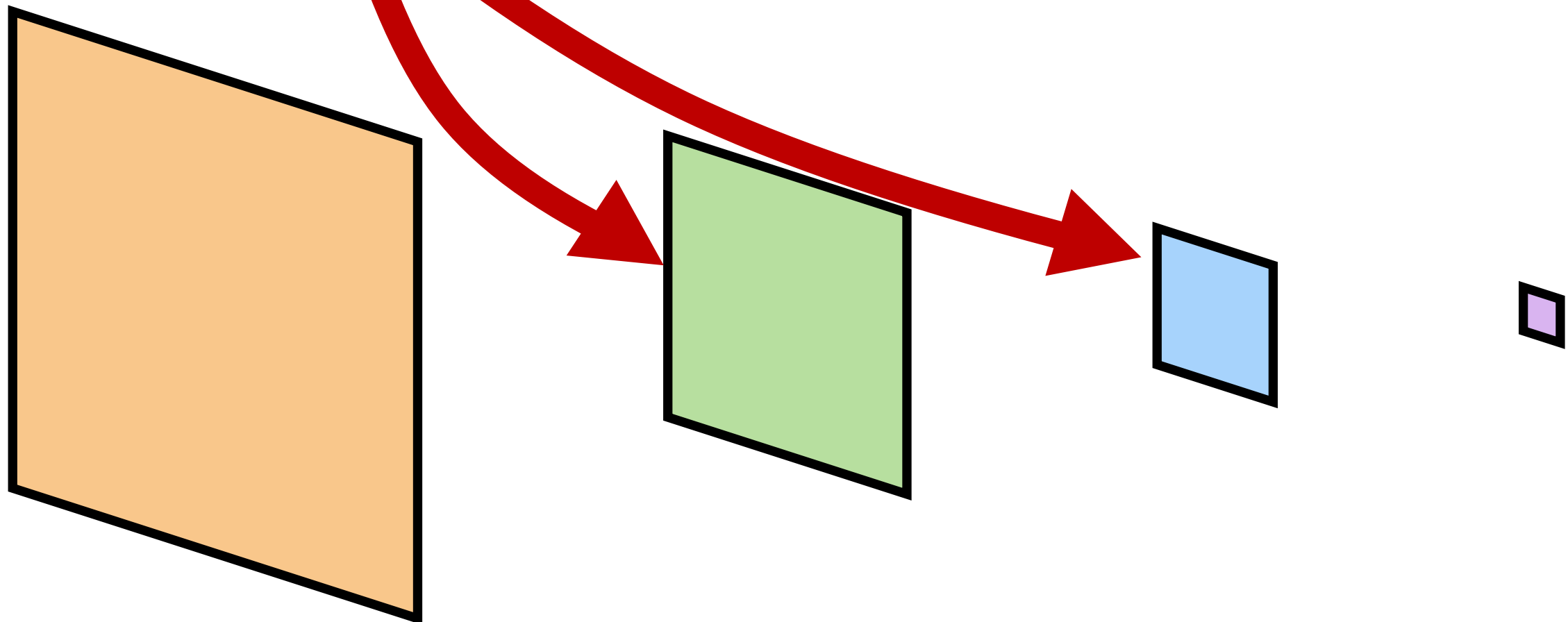
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

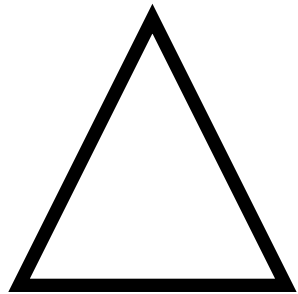
$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$





$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

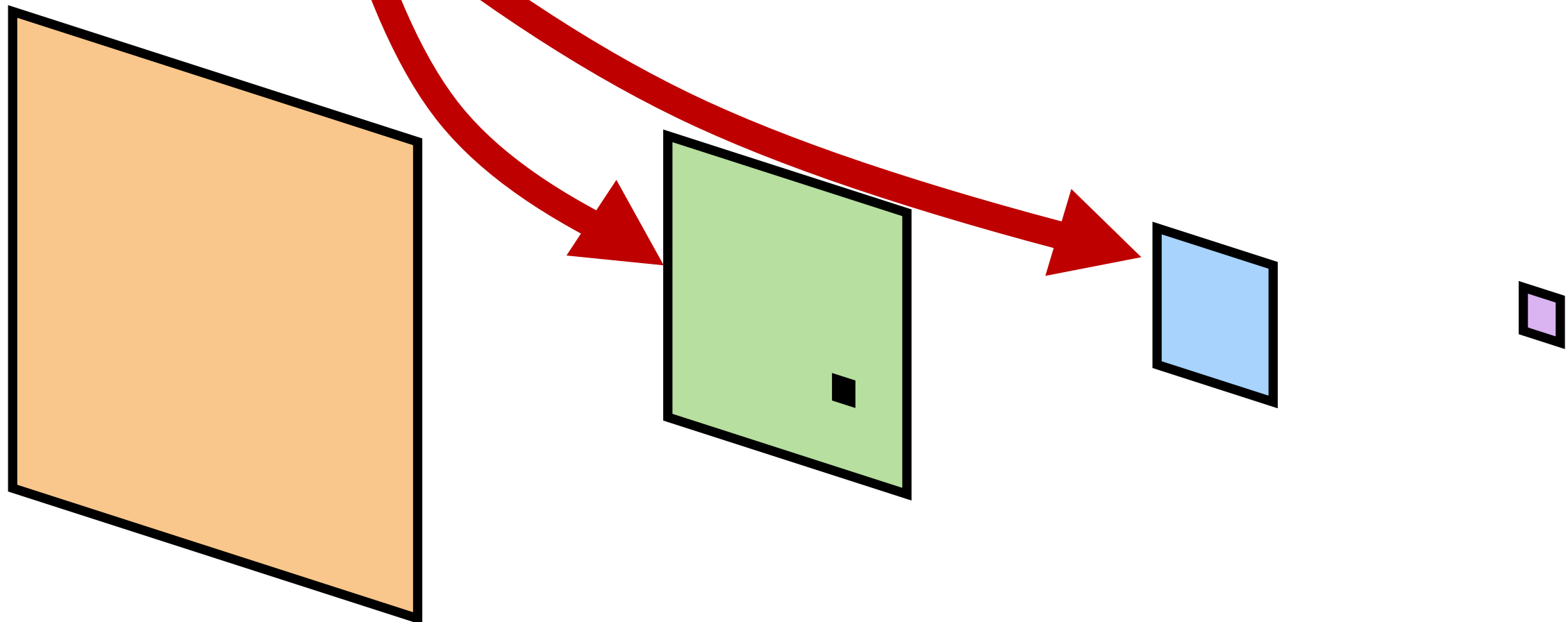
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

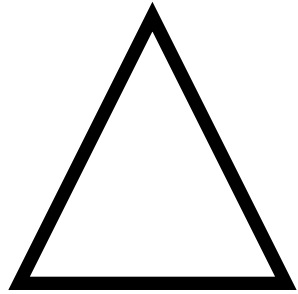
$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$





$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

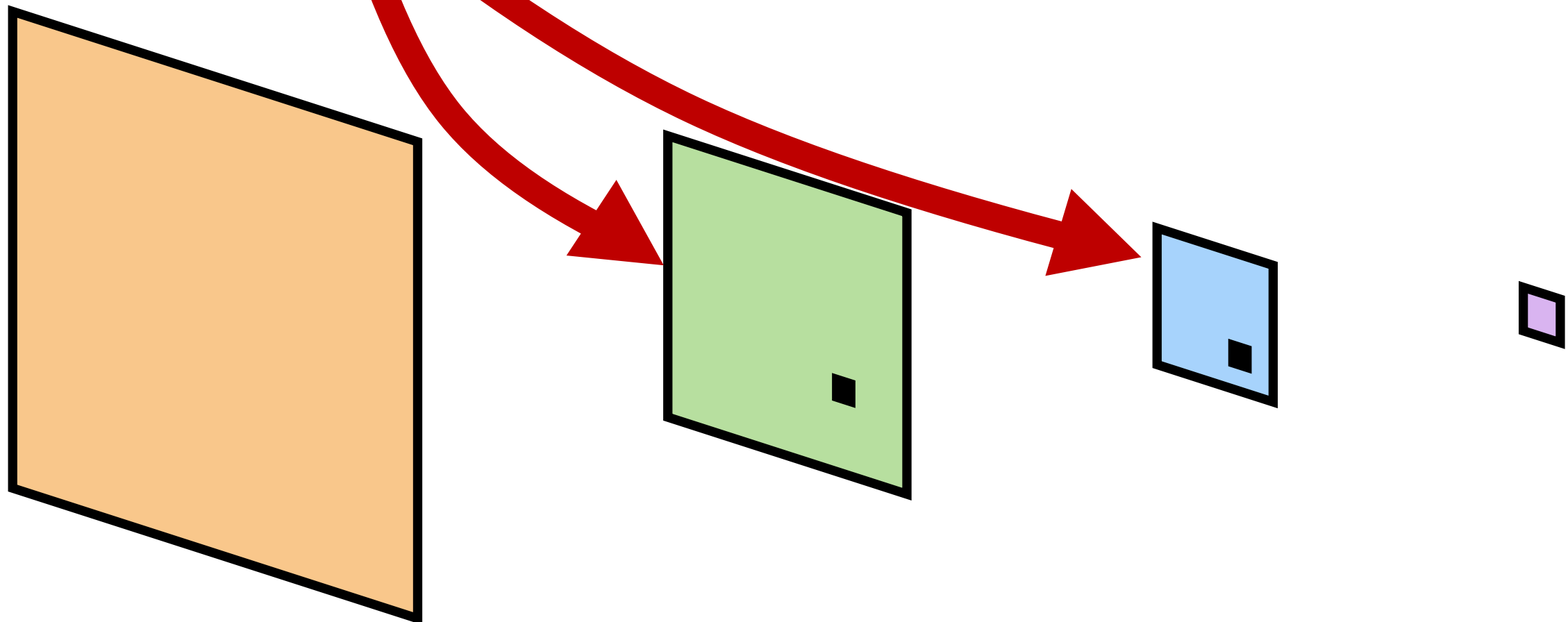
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

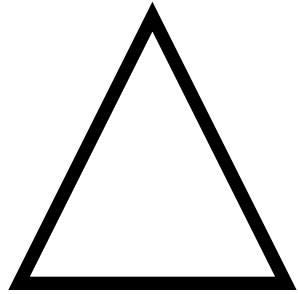
$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$





$$\rho = \max \left\{ \sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial w}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 + \left(\frac{\partial w}{\partial y}\right)^2} \right\}$$

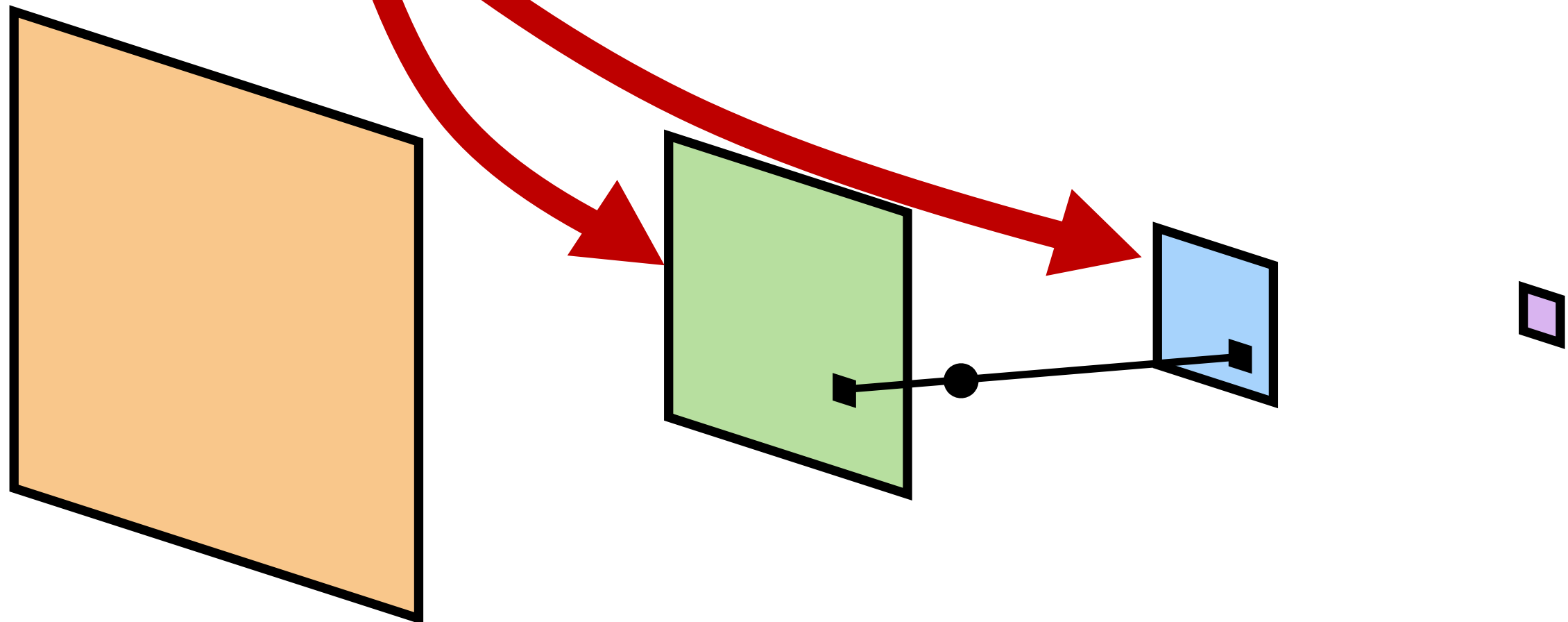
$$\lambda_{base}(x, y) = \log_2[\rho(x, y)]$$

$$\lambda'(x, y) = \lambda_{base}(x, y) + \text{clamp}(\text{bias}_{texobj} + \text{bias}_{shader})$$

<https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf>

п. 3.9.11

$$\lambda = \begin{cases} lod_{max}, & \lambda' > lod_{max} \\ \lambda', & lod_{min} \leq \lambda' \leq lod_{max} \\ lod_{min}, & \lambda' < lod_{min} \\ undefined, & lod_{min} > lod_{max} \end{cases}$$



512×512



GL_LINEAR

32×32



GL_LINEAR_MIPMAP_LINEAR

32×32



GL_LINEAR

512×512



GL_LINEAR

32×32



GL_LINEAR_MIPMAP_LINEAR

32×32



GL_LINEAR

Уровень оценивается для полигона целиком

512 × 512



GL_LINEAR

32 × 32



GL_LINEAR_MIPMAP_LINEAR

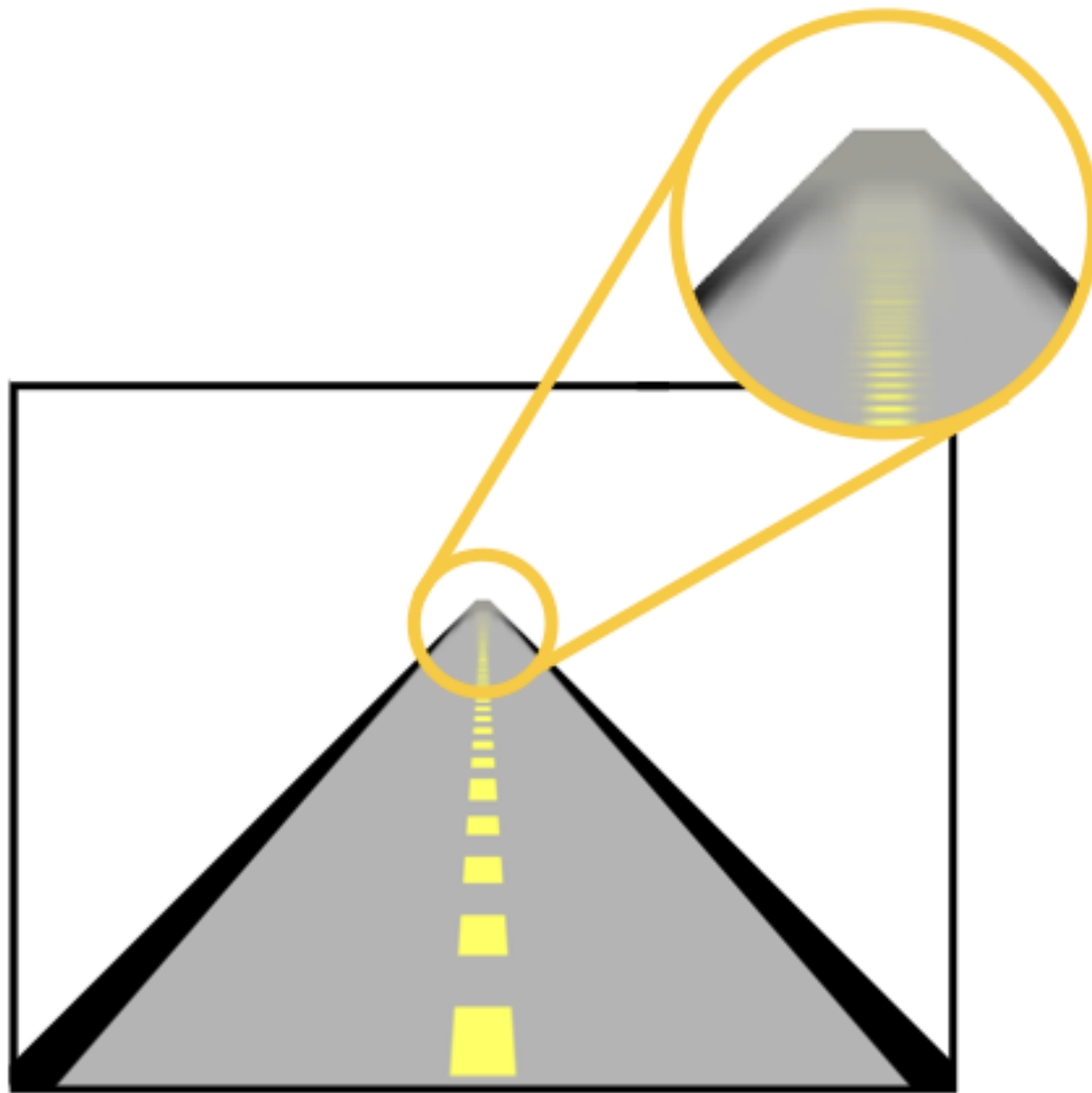
32 × 32



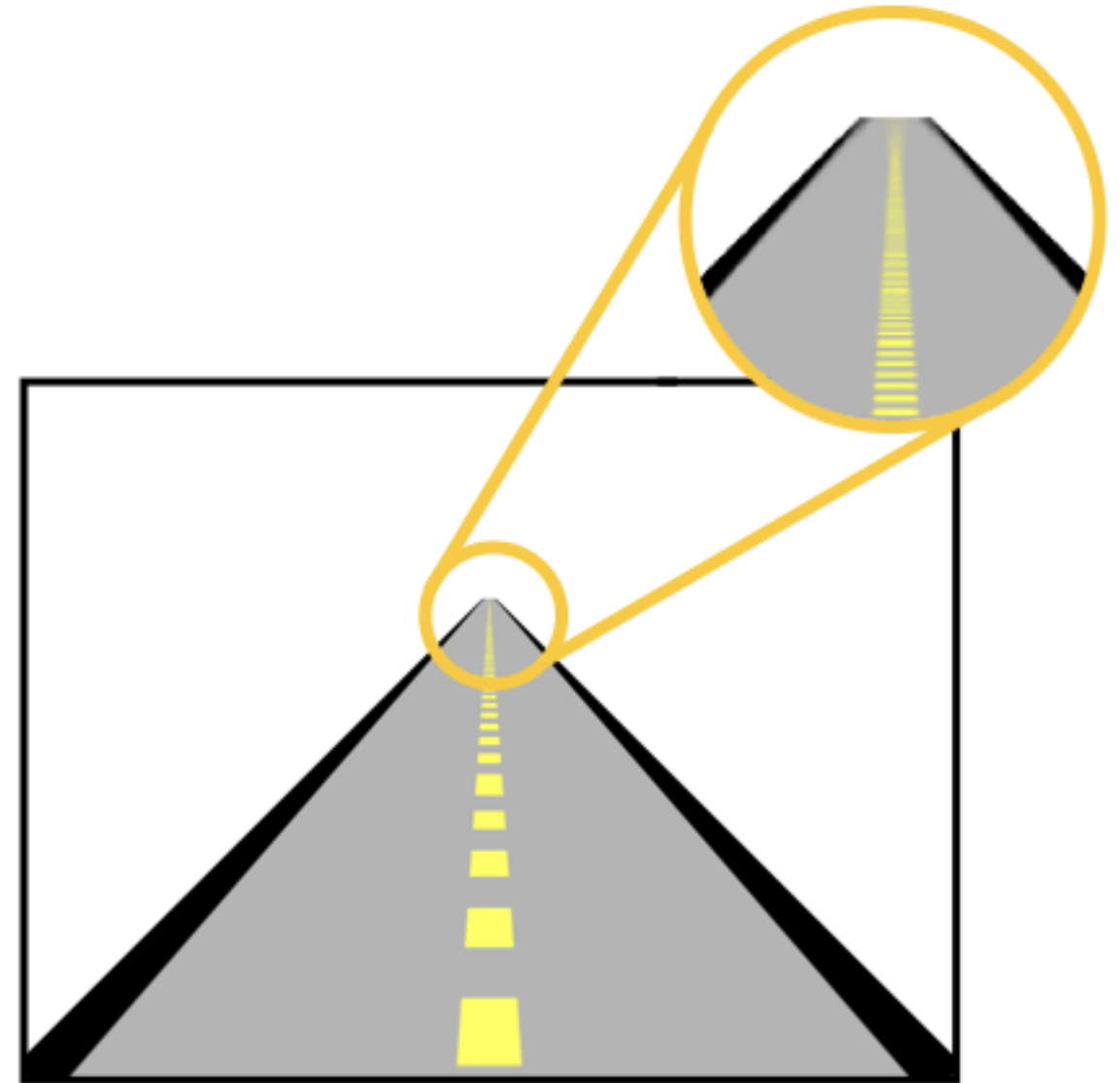
GL_LINEAR

Уровень оценивается для полигона целиком

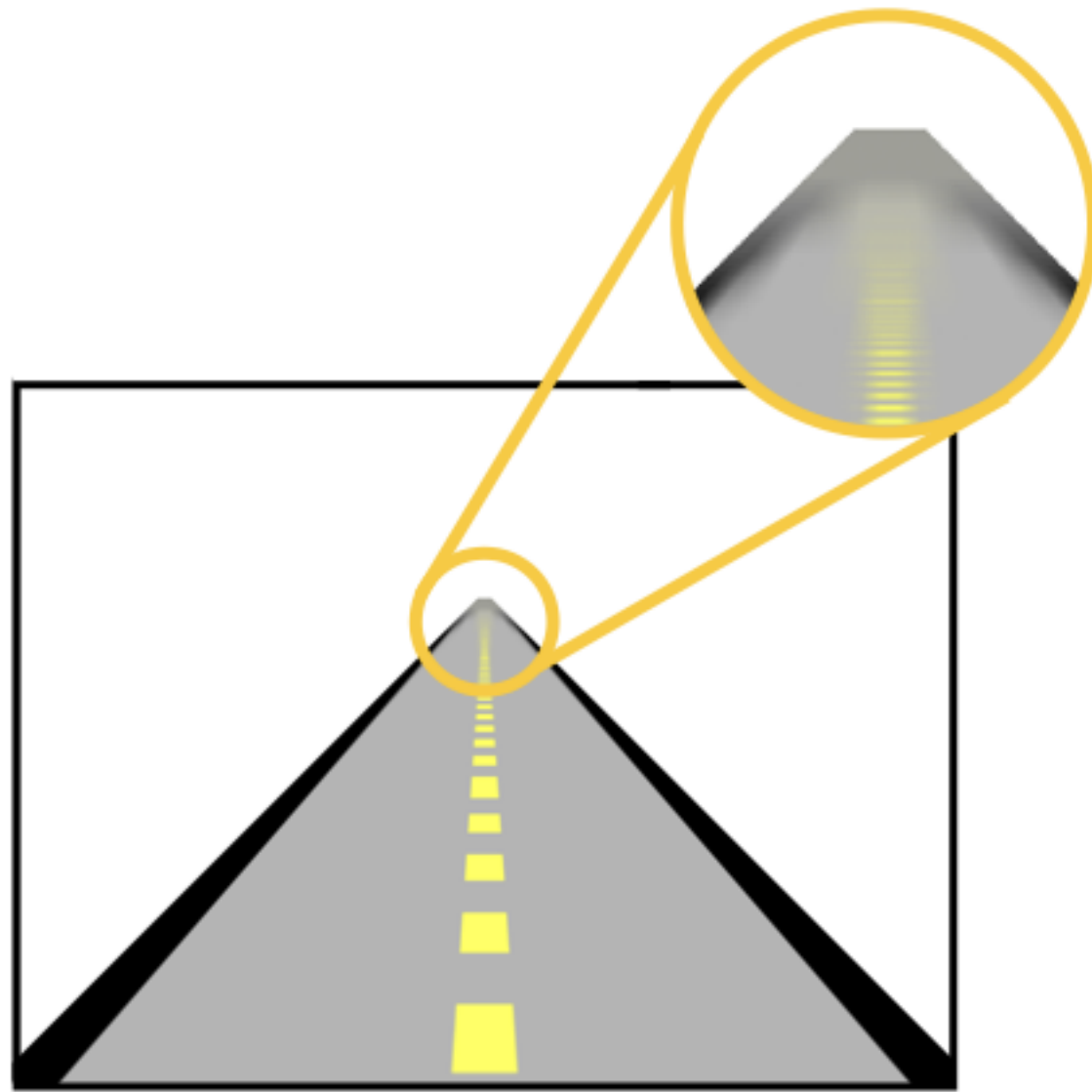
**Остаётся проблема, когда полигон длинный
и под острым углом к наблюдателю**



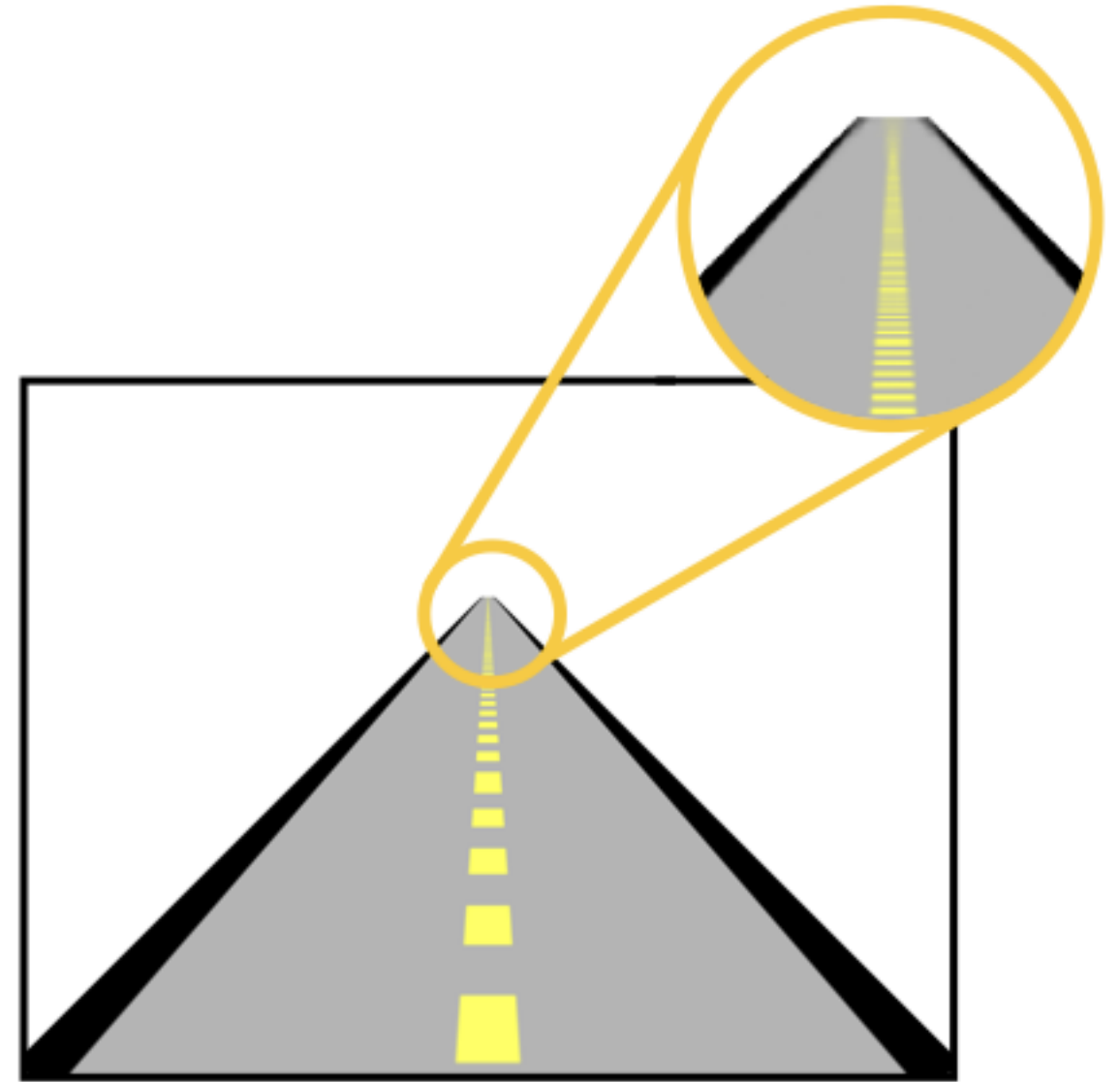
Билинейная
фильтрация



Анизотропная
фильтрация

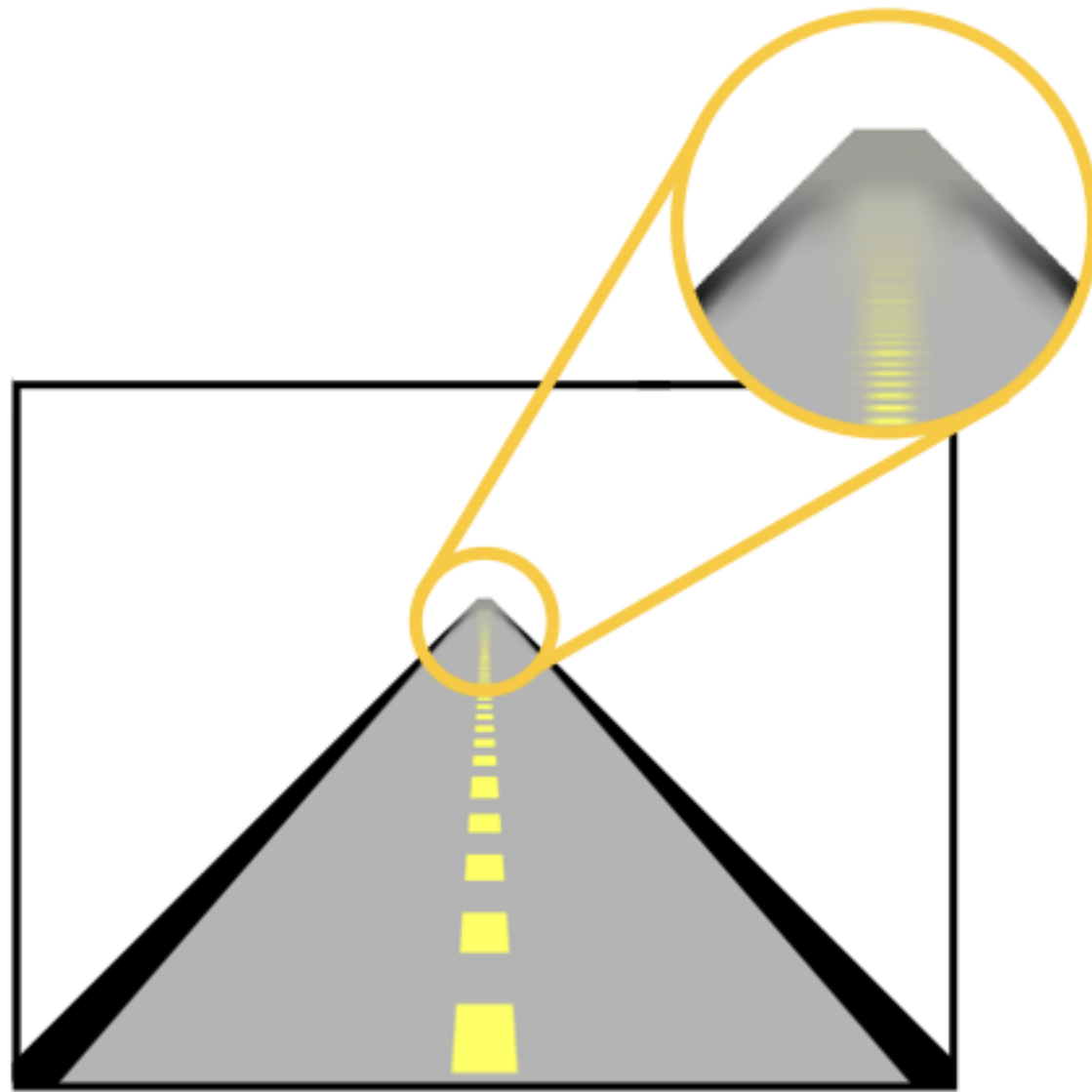


Билинейная
фильтрация

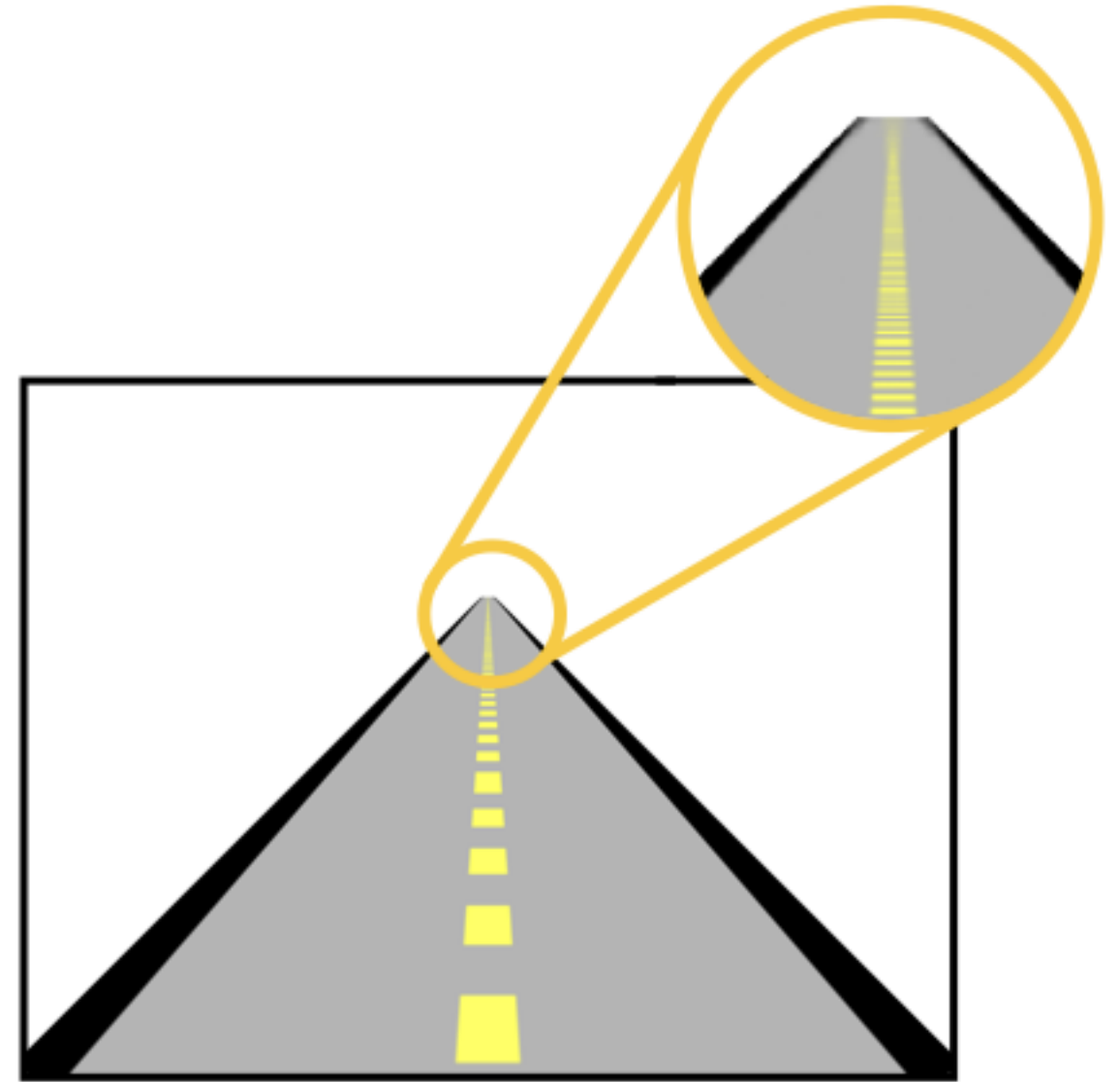


Анизотропная
фильтрация

**Уровень оценивается для каждого участка
полигона в отдельности**



Билинейная
фильтрация



Анизотропная
фильтрация

**Уровень оценивается для каждого участка
полигона в отдельности**

Медленнее, но качественнее

Мипмэппинг: анизотропная фильтрация 17 / 21

512x512



512x256



256x256



256x128



128x256



256x512

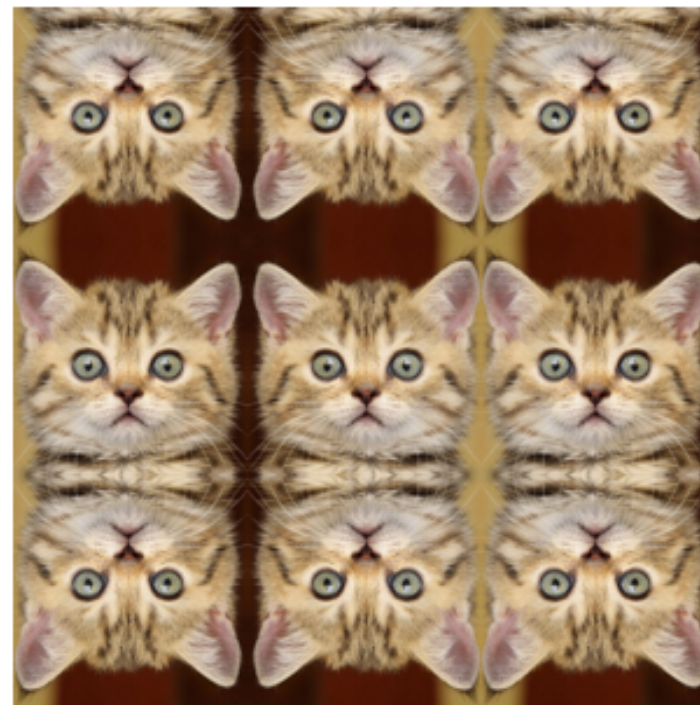
**Текстурные координаты меняются
в отрезке $[0; 1]$**

**Но на практике текстурные координаты могут
быть любыми**

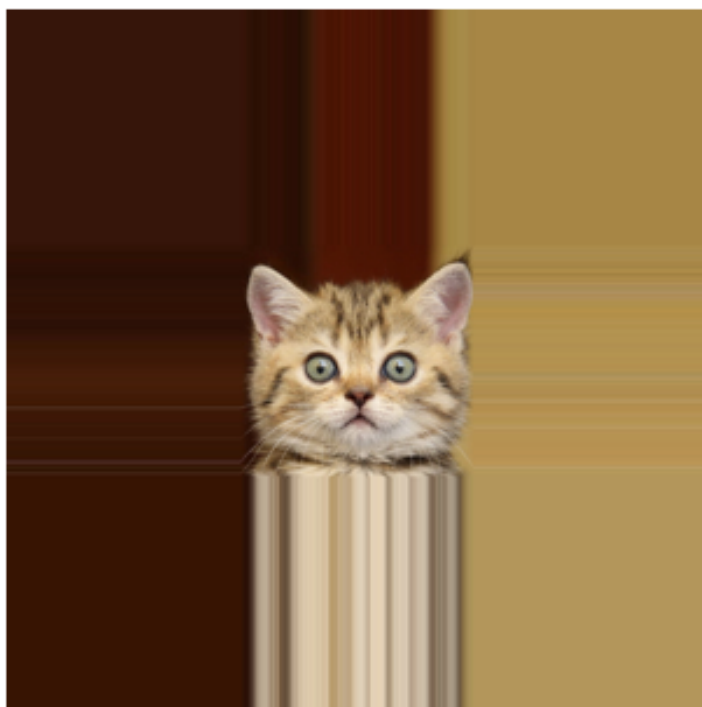
**Пересчёт произвольных координат
в отрезок $[0; 1]$ называется **режимом повторения****



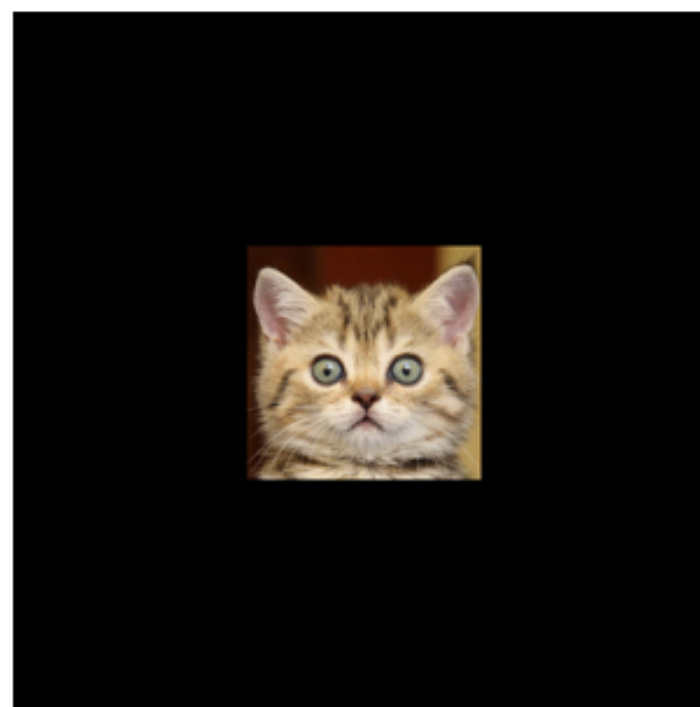
GL_REPEAT



GL_MIRRORED_REPEAT



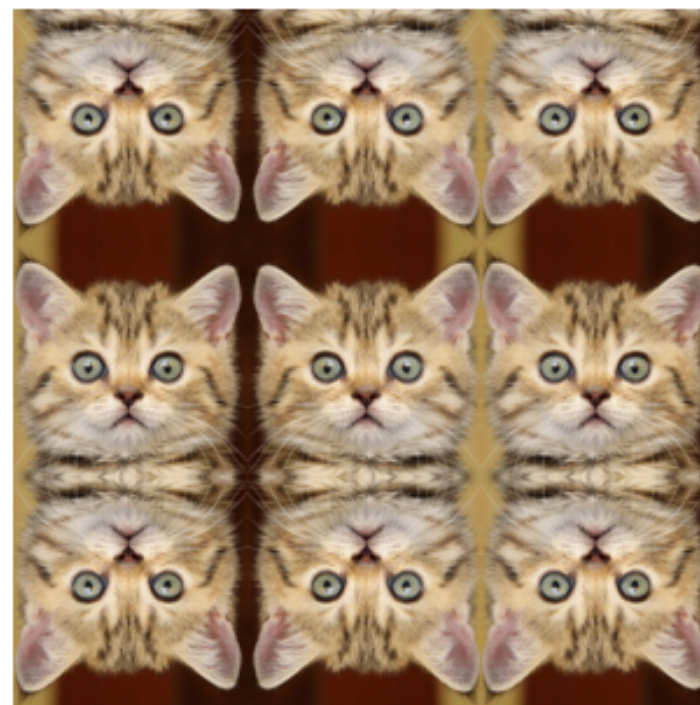
GL_CLAMP_TO_EDGE



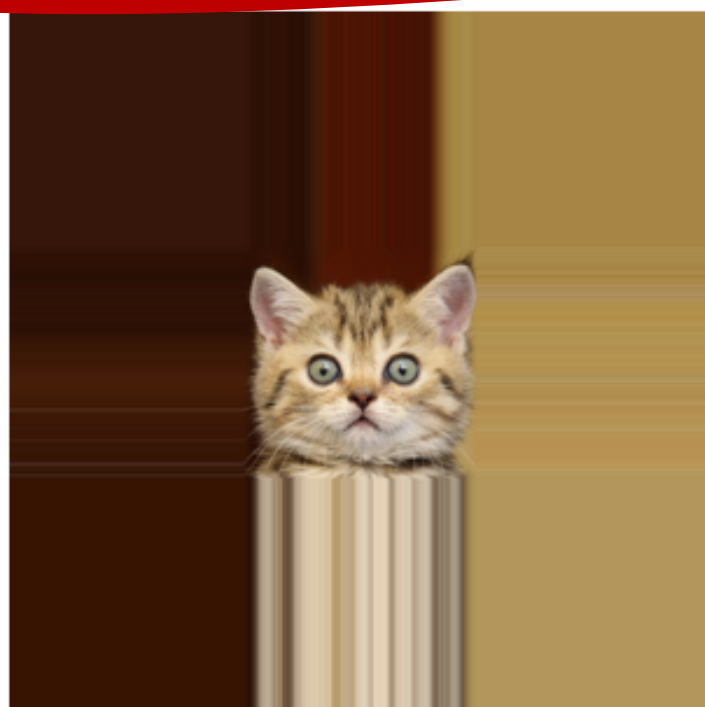
GL_CLAMP_TO_BORDER



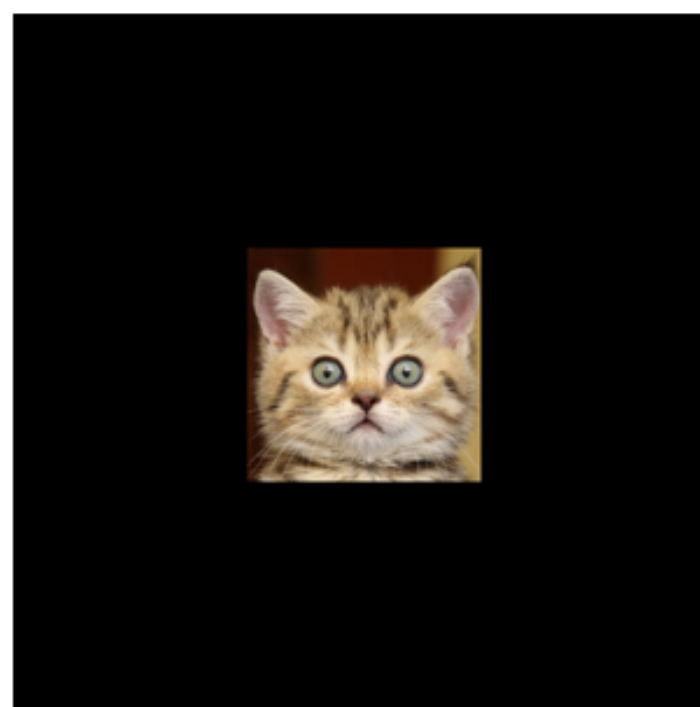
GL_REPEAT



GL_MIRRORED_REPEAT



GL_CLAMP_TO_EDGE



GL_CLAMP_TO_BORDER

Полезно для
ландшафтов



- 1. Формирование массива байтов
(загрузка из файла, процедурная генерация)**
- 2. Генерация текстурного объекта**
- 3. Загрузка данных в видеопамять**
- 4. Настройка параметров
(режимы минификации, магнификации,
повторения, ...)**
- 5. Передача в шейдер (uniform-переменная)**
- 6. Выборка (семплинг)**

1. Создание дополнительного буфера кадра
2. Генерация текстурного объекта
3. Присоединение текстуры к буферу кадра в качестве буфера цвета
4. Настройка параметров
(режимы минификации, магнификации, повторения, ...)
5. Активация созданного буфера кадра
6. Визуализация сцены
7. Активация стандартного буфера кадра
8. Визуализация сцены
9. Передача в шейдер (uniform-переменная)
10. Выборка (семплинг)